



Free Type Foundry

Editor de tipografías 100% en el navegador

Sube una fuente, edita cada letra en su recuadro y exporta tu propia fuente TTF instalable

www.freeanimationpower.org

Documentación técnica completa · Versión 1.0 · Septiembre 2026

Autor: Eduardo Fierro Duque · Ecosistema Free Animation Power · Licencia GPL v3.0

Contenido

1. ¿Qué es Free Type Foundry?
2. Características principales
3. Ejecución de la aplicación
4. Estructura de archivos
5. El tablero de letras (planilla)
6. Guías métricas de cada recuadro
7. Herramientas de edición
8. El motor geométrico Bézier
9. Modelo de datos de un glifo
10. Importación de fuentes
11. Exportación a TTF
12. Proyectos .ftf
13. Deshacer y rehacer
14. Atajos de teclado
15. Estandarización de tamaños
16. Pruebas automatizadas
17. Dependencias
18. Hoja de ruta
19. Licencia y créditos

1. ¿Qué es Free Type Foundry?

Free Type Foundry es una **fundición tipográfica digital que corre 100% en el navegador**. Su concepto central: todas las letras y símbolos de una fuente viven en un tablero de recuadros — uno por carácter — y cada recuadro es un mini-editor vectorial independiente.

No requiere instalación ni servidor: se abre con doble clic en `index.html`. Importar, editar y exportar ocurre íntegramente en el computador del usuario; ningún dato se sube a servidores externos.

Flujo típico:

1. Empieza con la fuente en blanco (caracteres ASCII y Latín-1 pre-creados) o sube una fuente existente para modificarla.
2. Edita cada letra en su recuadro: mueve puntos, añade curvas, dibuja trazos.
3. Exporta el TTF y lo instalas en tu sistema operativo.

2. Características principales

Función	Detalle
Importación	TTF, OTF, WOFF y WOFF2; cualquier upm se normaliza a 1000
Tablero	Un recuadro por letra, con círculo naranja (letra) y código Unicode
Edición de puntos	Puntos on-curve (cuadrados) y off-curve (círculos) arrastrables
Añadir puntos	Clic sobre un trazo inserta un punto que preserva la forma
Dibujo libre	Trazo con presión de lápiz convertido en contorno vectorial suavizado
Líneas rectas	Contornos polígono punto a punto
Curvas	Conversión recta ↔ curva por segmento y suavizado de esquinas
Borrado seguro	Vista previa en rojo del contorno antes de eliminarlo
Avance	Advance width editable por glifo (línea azul punteada)
Guías	Línea base, altura x, mayúsculas, ascendente y descendente
Zoom global	Slider que agranda todas las celdas a la vez
Exportación	TTF estándar con metadatos completos
Proyectos	Guardar/abrir como .ftf (JSON)
Deshacer	Historial de 200 acciones (Ctrl+Z / Ctrl+Y)

3. Ejecución de la aplicación

La app es estática (HTML + CSS + JS). Solo se necesita internet la primera vez para cargar `opentype.js` desde CDN.

- **Opción A (directo):** doble clic en `index.html`.
- **Opción B (servidor local):** `php -S 127.0.0.1:8899 -t .` y abrir `http://127.0.0.1:8899/index.html`.
- **Opción C (producción):** se empaqueta como `tools/font/index.php` dentro del hub Free Animation Power, detrás del email-gate de sesión.

4. Estructura de archivos

```
FREE-TYPE-FOUNDRY/
├─ index.html           Interfaz completa
├─ style.css            Tema corporativo (negro + naranja #ff3300)
├─ favicon.png          Logo 64x64
├─ LOGO FREE TYPE FOUNDRY.png Logo oficial 1254x1254
├─ js/
│  ├─ geometry.js       Motor geométrico Bézier (cero dependencias)
│  ├─ importer.js       Parse TTF/OTF/WOFF → modelo interno
│  ├─ exporter.js       Modelo interno → TTF + proyectos .ftf
│  ├─ editor.js         FTGridEditor: render e interacción del tablero
│  └─ main.js           Estado global, UI, undo/redo, atajos
├─ tests/              Pruebas Node + harness de navegador
├─ docs/               Documentación (HTML y PDF)
├─ GUIA-RAPIDA.md      Guía simplificada
└─ README.md           Documentación completa (markdown)
```

Responsabilidades de cada módulo

geometry.js — Puro, funciona en Node y navegador. Define `normalizeContour()` (resuelve puntos implícitos de TrueType), `eachSegment()`, `commandsToContours()/contoursToCommands()`, `cubicToQuadratic()` (cúbicas → cuadráticas con tolerancia 0.5), `insertPointOnSegment()` (división por de Casteljau), `removePointAt()`, `smoothAt()`, `setSegmentCurved()`, `strokeToContour()` (trazo → contorno: offset izquierdo/derecho, Ramer–Douglas–Peucker y Chaikin) y `glyphNameFor()`.

importer.js — `parseFont()` lee el binario con `opentype.parse()`, normaliza a 1000 UPM, conserva solo glifos con Unicode, convierte comandos a contornos internos y devuelve `{familyName, glyphs, count}`. `defaultGlyphSet()` crea la fuente en blanco (códigos 32–126 y 161–255, avance 620).

exporter.js — `buildFont()` crea `.notdef` (glifo 0), ordena por Unicode, construye cada glifo con su Path Q y escribe metadatos (familia, diseñador, fabricante, licencia, versión, copyright). `toArrayBuffer()` produce el binario TTF. También `serializeProject()/deserializeProject()`.

editor.js — `FTGridEditor`: construye celdas DOM con render diferido (IntersectionObserver), pinta guías, glifo en negro intenso, puntos, selección, borrador y trazo en curso. Transformación mundo↔celda con eje Y volteado (evita letras cabeza abajo). Hit-testing de puntos (7px) y segmentos (25 muestras en curvas). Arrastre con Pointer Events y captura de puntero. `_commit()` alimenta el historial con snapshots.

main.js — Estado global, planilla con filtros y buscador, undo/redo (200 entradas), topbar (nueva/subir/guardar/abrir/exportar), atajos y slider de celda.

5. El tablero de letras (planilla)

Cuadrícula de **6 columnas centradas** que fluye hacia abajo. Cada celda contiene:

- **Lienzo blanco**: la letra en negro intenso con sus puntos de edición visibles.
- **Círculo naranja** (arriba a la izquierda): la letra o símbolo de esa celda, en negro.
- **Código Unicode** (abajo a la derecha): valor hexadecimal sobre pastilla naranja translúcida.
- **Línea azul punteada**: el avance del glifo.

Filtros

Filtro	Rango	Uso
Básico	U+0020 – U+007E	Mayúsculas, minúsculas, números y signos comunes
Latín-1	U+00A1 – U+00FF	Acentos y signos: á é í ó ú ñ ¿ ¡
Puntos	Signos de puntuación	, . ; : ! ? ...
Todo	Todos	Fuente completa (importada)

El buscador acepta carácter (`A`), código (`0041`) o nombre del glifo. El botón **+ Glifo** añade caracteres nuevos; si ya existen, los selecciona.

Tamaño de celda

El slider **Celda** (80–260px) agranda o reduce todas las celdas a la vez manteniendo la escala relativa. Por teclado: `+`, `-` y `0` (reset 140px).

6. Guías métricas de cada recuadro

Guía	Valor (y)	Color	Significado
Ascendente	+800	Gris claro	Tope de b, d, h

Altura mayúsculas	+700	Verde	Tope de A, B, C
Altura x	+500	Azul	Tope de a, e, o
Línea base	0	Roja	Donde se apoyan las letras
Descendente	-200	Gris claro	Tope de g, p, q, y
Origen	x = 0	Gris claro	Margen izquierdo
Avance	x = avance	Azul punteado	Inicio de la siguiente letra

7. Herramientas de edición

Herramienta	Tecla	Comportamiento
Mover	V	Arrastra puntos on-curve (cuadrados) y off-curve (círculos naranjas). Mover un punto on-curve arrastra consigo sus controles adyacentes. Arrastrar un trazo mueve el contorno completo. Clic sobre un trazo lo selecciona para Curva/Recta.
Punto	A	Clic sobre un trazo inserta un punto preservando la forma (en curvas divide la cuadrática exactamente en el clic).
Borrar	D	Clic sobre un punto lo elimina y re-normaliza el contorno.
Dibujar	B	Trazo libre con presión. Al soltar se convierte en contorno cerrado y suavizado. Grosor configurable (10–200).
Línea	L	Contorno de segmentos rectos clic a clic. Doble clic o Enter cierra; Esc cancela.
Contorno	E	Borra un contorno completo. Vista previa en rojo punteado; solo borra si el clic cae exactamente sobre el trazo (4.5px).

Botones estructurales

- **Suavizar:** convierte la esquina del punto seleccionado en curva continua.
- **Curva:** añade un control en el punto medio del segmento seleccionado.
- **Recta:** elimina el control del segmento seleccionado.
- **Vaciar:** borra todos los contornos del glifo seleccionado (con confirmación).

El campo **Avance** de la barra inferior define el desplazamiento tras la letra y se refleja como línea azul punteada; es parte del historial de deshacer.

8. El motor geométrico Bézier

Por qué cuadráticas

TrueType (.ttf) representa contornos con **Bézier cuadráticas**: segmentos entre puntos on-curve con un punto off-curve de control opcional. CFF/OTF usa cúbricas. Free Type Foundry trabaja siempre con cuadráticas, por lo que lo que se ve en pantalla es exactamente lo que se exporta. Al importar OTF, cada cúbrica se convierte a cuadráticas con tolerancia de 0.5 unidades.

Regla de oro de TrueType

Dos puntos off-curve consecutivos implican un punto on-curve en su punto medio.

`normalizeContour()` aplica esta regla y garantiza secuencias válidas: on-curve, (off-curve), on-curve...

Del trazo al contorno

1. Registro de muestras del puntero con su presión (0–1).
2. Offset izquierdo/derecho perpendicular, con ancho proporcional a la presión (`grosor × (0.25 + 0.75 × presión)`).
3. Simplificación con Ramer–Douglas–Peucker (tolerancia 1.5).
4. Suavizado de esquinas con Chaikin (patrón alternado on/off-curve).
5. Normalización y añadido del contorno al glifo.

9. Modelo de datos de un glifo

```
{
  unicode: 65,
  name: 'A',
  advanceWidth: 620,
  contours: [
    [
      { x: 100, y: 0,   on: true  },
      { x: 180, y: 350, on: false },
      { x: 260, y: 0,   on: true  },
      { x: 260, y: 700, on: true  }
    ]
  ]
}
```

El sentido del giro de cada contorno define el relleno en la convención non-zero de TrueType. El editor pinta con relleno negro sólido even-odd, por lo que el usuario no gestiona direcciones manualmente.

10. Importación de fuentes

1. **Subir TTF/OTF** abre el selector (`\.ttf`, `\.otf`, `\.woff`, `\.woff2`).
2. `FileReader` carga el binario como `ArrayBuffer`.
3. `opentype.parse()` interpreta las tablas (cmap, glyf/CFF, hmtx, head...).
4. Normalización a 1000 UPM multiplicando coordenadas y avances por `1000/unitsPerEm`.
5. Solo se importan glifos con Unicode asignado, sin duplicados; `.notdef` se regenera al exportar.
6. El tablero se rellena y se selecciona la primera letra dibujada.

Nota: importar no sube el archivo a ningún servidor: todo el parse ocurre en memoria, en el navegador.

11. Exportación a TTF

1. **Exportar TTF** (botón naranja).
2. Se construye la fuente: `.notdef` como glifo 0 + glifos ordenados por Unicode con sus comandos M/L/Q/Z.
3. Metadatos en la tabla name: familia, subfamilia, nombre completo, PostScript, diseñador (Eduardo Fierro Duque), fabricante y URL (Free Animation Power), licencia GPL v3.0, versión y copyright.
4. `toArrayBuffer()` genera el binario que se descarga como `<nombre>.ttf`.

Instalación

- **Windows:** doble clic → Instalar.
- **macOS:** doble clic → Instalar fuente.
- **Linux:** copiar a `~/fonts` o `/usr/share/fonts` y ejecutar `fc-cache -f`.

12. Proyectos .ftf

El formato `.ftf` es un JSON con todo el estado editable. Cada punto se serializa como `[x, y, on]`. Al abrir se valida el campo `app: "freetypefoundry"`.

```
{
  "app": "freetypefoundry",
  "version": 1,
  "savedAt": "2026-09-24T12:00:00.000Z",
  "familyName": "Mi Fuente",
  "glyphs": [
    { "unicode": 65, "name": "A", "advanceWidth": 620,
      "contours": [ [ [100, 0, 1], [180, 350, 0], [260, 0, 1] ] ] }
  ]
}
```

```
]
}
```

13. Deshacer y rehacer

- Historial de 200 entradas con snapshots completos (contornos + avance) del glifo afectado.
- `Ctrl+Z` deshace · `Ctrl+Y` rehace · botones en la toolbar.
- Cada acción muestra un toast con su nombre.
- El historial sobrevive al cambio de celda: deshacer afecta al glifo correcto aunque ya no esté seleccionado.

14. Atajos de teclado

Tecla	Acción
V	Mover
A	Añadir punto
D	Borrar punto
B	Dibujar
L	Línea
E	Borrar contorno
Ctrl+Z / Ctrl+Y	Deshacer / Rehacer
Enter	Cerrar contorno (Línea)
Esc	Cancelar contorno / quitar selección
+ / -	Agrandar / reducir celdas
0	Tamaño de celda por defecto (140px)

15. Estandarización de tamaños

Todas las celdas comparten la misma ventana de coordenadas (x: -100 a 1080 · y: -300 a 860) y la misma escala. Al importar, todo se normaliza a 1000 UPM. Resultado: una letra que toca la altura de mayúsculas tiene exactamente el mismo tamaño físico en la fuente final que cualquier otra, evitando el problema clásico de las fuentes caseras con tamaños disparejos.

16. Pruebas automatizadas

Requiere Node.js ≥ 18 (solo para tests; la app no lo necesita).

```
npm install
node tests/geometry.js      19 tests del motor geométrico
node tests/export.js        roundtrip: TTF generado → re-leído → verificado
node tests/import-real.js   importación de arial.ttf real (upm 2048 → 1000)
node tests/flip.js          inspección de comandos exportados
```

El harness de navegador `tests/harness.html` (14 pruebas integrales) se abre en cualquier navegador o con `php -S 127.0.0.1:8899 -t .` y visitando `/tests/harness.html`.

17. Dependencias

- **opentype.js 1.3.4** (MIT, CDN jsdelivr): única dependencia de runtime. Tercera excepción CDN del ecosistema Free Animation Power (tras p5.js en Glitch y jsPDF en Storyboard).
- Todo lo demás: vanilla JS, CSS y HTML sin frameworks.

18. Hoja de ruta

- Edición de kerning por pares (tabla kern).
- Múltiples pesos/estilos por proyecto.
- Ajuste de side bearings arrastrando en pantalla.
- Plantilla base con proporciones clásicas (roman, cursiva).
- Previsualización de frase de prueba en vivo.
- Exportar WOFF2.
- Integración al hub (tools/font/index.php single-file).

19. Licencia y créditos

- **Código:** GNU General Public License v3.0 (GPLv3).
- **Autor:** Eduardo Fierro Duque — fierroduque.com
- **Ecosistema:** www.freeanimationpower.org
- **GitHub:** github.com/freeanimationpower

Las fuentes creadas con esta herramienta son del usuario: la GPL cubre el editor, no lo que se dibuja con él.

